Construcción de software y toma de decisiones (Gpo 501)

Lectura: Metodología para diseñar casos de pruebas a partir de casos de uso

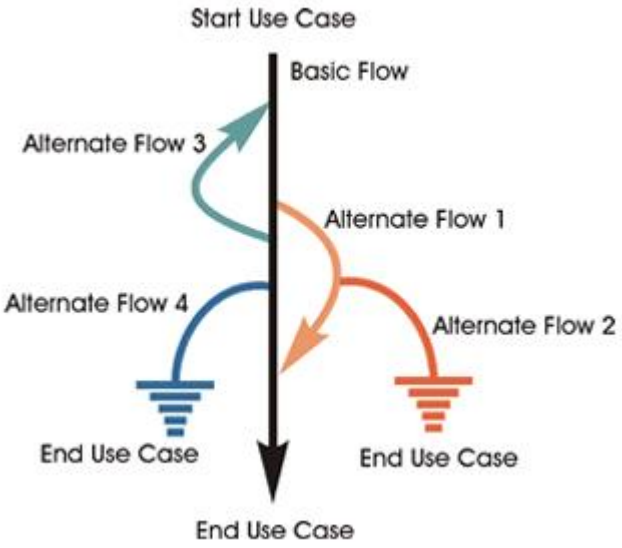
Carlos Arturo Gómez Ayala - A01711027
ITESM Campus Querétaro
Profesor: Enrique Alfonso Calderón Balderas
Co-Titular: Alejandro Fernández Vilchis
Co-Titular: Denisse Lizbeth Maldonado Flores

Fecha: 1/5/2026

Table 1: Format for a Use-Case Textual Description

Use Case Section	Description
Name	An appropriate name for the use case (see Leslee Probasco's article in the March issue of *The Rational Edge*).
Brief Description	A brief description of the use case's role and purpose.
Flow of Events	A textual description of what the system does with regard to the use case (not how specific problems are solved by the system). The description should be understandable to the customer.
Special Requirements	A textual description that collects all requirements, such as non-functional requirements, on the use case, that are not considered in the use-case model, but that need to be taken care of during design or implementation.
Preconditions	A textual description that defines any constraints on the system at the time the use case may start.
Post conditions	A textual description that defines any constraints on the system at the time the use case will terminate.

Los casos de uso sirven para definir lo que debe hacer un sistema desde el punto de vista del usuario o de otro sistema que interactúa con él. En UML se representan con óvalos y los actores con figuras humanas. Cada caso de uso representa una parte importante de la funcionalidad del sistema, y los actores son quienes usan o se comunican con esa función.



Un caso de uso no solo es el diagrama, también necesita una descripción escrita. Normalmente incluye nombre, descripción breve, flujo de eventos, requisitos especiales, precondiciones y postcondiciones. La parte más importante para las pruebas es el flujo de eventos, porque ahí se ve paso a paso qué hace el usuario y cómo responde el sistema.

El flujo básico es el camino normal, o sea cuando todo funciona bien. Por ejemplo, en un sistema de registro de cursos, el estudiante inicia sesión, elige crear horario, revisa los cursos disponibles, selecciona sus materias, manda el horario y el sistema confirma el registro. Ese sería el caso ideal o exitoso. De los escenarios se crean los casos de prueba. Cada caso debe tener entradas, condiciones de ejecución y un resultado esperado. Para organizar esto se puede usar una matriz de pruebas, donde se colocan datos como usuario, contraseña, cursos seleccionados y condiciones del sistema. Antes de usar datos reales, se pueden marcar las condiciones con V, I y N/A, para indicar si son válidas, inválidas o si no aplican. Después esas marcas se sustituyen por datos concretos para poder ejecutar las pruebas.
La idea principal es que las pruebas pueden empezar desde que se definen los casos de uso, no hasta que el sistema ya esté programado. Esto ayuda a encontrar errores antes, organizar mejor el trabajo y comprobar que el sistema cumpla con sus requisitos.

Putting It All Together

In current practice, use cases are associated with the front end of the software development lifecycle and test cases are typically associated with the latter part of the lifecycle. By leveraging use cases to generate test cases, however, testing teams can get started much earlier in the lifecycle, allowing them to identify and repair defects that would be very costly to fix later, ship on time, and ensure that the system will work reliably.

Using the clearly-defined methodology I've outlined above for generating test cases, developers can simplify the testing process, increase efficiency, and help ensure complete test coverage.